Operations Research

Scheduling of a Flexible Flow Line with Limited Buffers and Bypass

Project Report — Chapter 16.3 (Pinedo)

Chapter	16.3 — Flexible Flow Line Loading (FFLL)
Source	Pinedo, Scheduling: Theory, Algorithms, and Systems
Topics	Problem formulation, MIP, FFLL algorithm, IG-LS improvement

Table of Contents

1. Problem Statement & Graham Classification

1.1 Problem Description

The problem considers a **Flexible Flow Line (FFL)** — a production environment consisting of multiple stages arranged in series, where each stage contains one or more machines operating in parallel. The specific context in Section 16.3 is the assembly of Printed Circuit Boards (PCBs), originally implemented at IBM.

The key characteristics of the environment are:

Jobs: Each job represents a batch of relatively small identical items (e.g., PCBs). A set of jobs that together represent one full day's production mix is called a Minimum Part Set (MPS). The manufacturing process is repetitive, so the MPS cycles through the system continuously.

Processing: Each job must be processed at every stage on exactly one machine. Not all machines at a stage are necessarily identical — some may handle only specific job types.

Bypass: A job may not require processing at every stage. If a job skips a stage, the material handling system transports it past that stage. This is modeled by setting the processing time of that job at the bypassed stage to zero ($p_{ij} = 0$).

Limited Buffers: Between stages, buffer space is finite. If a buffer is full, the material handling system comes to a standstill — or, if recirculation is possible, the job bypasses the stage and recirculates. This makes buffer management critical to system performance.

Cyclic Schedule: Because production is repetitive, the goal is to find a cyclic schedule — the best ordering and timing of jobs within one MPS cycle — that can be repeated indefinitely.

1.2 Objectives & Alternative Objective Functions

The FFLL algorithm pursues two objectives simultaneously:

Primary objective — Maximize throughput, which in cyclic scheduling is equivalent to minimizing the cycle time of one MPS. The cycle time cannot be less than the workload at the bottleneck machine, so the algorithm tries to approach this lower bound.

Secondary objective — Minimize Work-In-Process (WIP), i.e., the number of jobs residing in the system at any point in time. Since buffer space is limited, reducing WIP lowers the probability of buffers becoming full and causing blockage.

Alternative objective functions include:

- Minimizing weighted tardiness — relevant when jobs have due dates and different priorities.
- Minimizing average flow time — reducing the average time a job spends in the system.
- Minimizing the number of tardy jobs — relevant in customer-facing production settings.
- Minimizing maximum lateness — relevant when due date violations have hard consequences.

1.3 Graham Notation (α | β | γ)

The standard three-field notation for this problem is:

$$\text{FFs} \mid \text{bypass}, b_i < \infty \mid C_{max}$$

Field	Symbol	Meaning
α (machine environment)	FFs	Flexible Flow Shop — stages in series, parallel machines per stage
β (constraints)	bypass	Jobs may skip stages (zero processing time at a stage)
	b■ < ∞	Finite buffer capacity between stages
γ (objective)	C_max	Minimize makespan / cycle time of the MPS

The problem is **NP-hard**: even the two-machine flow shop with blocking is NP-hard, and adding bypass and parallel machines further increases complexity. This motivates the use of heuristic methods such as the FFLL algorithm rather than exact optimization.

2. MIP Formulation & Critical Assessment

The chapter does not explicitly present a Mixed Integer Program (MIP). We construct one from the problem structure, mirroring the three decisions of the FFLL algorithm: machine assignment, job sequencing, and timing.

2.1 Indices, Parameters & Decision Variables

Indices and Parameters:

- **n**: Number of jobs in the MPS, indexed j = 1, ..., n
- **m**: Total number of machines, indexed i = 1, ..., m
- **S**: Number of stages, indexed s = 1, ..., S
- **M_s**: Set of machines at stage s
- **p_ij**: Processing time of job j on machine i (0 if job bypasses that stage)
- **b_s**: Buffer capacity between stage s and stage s+1
- **L**: Sufficiently large constant (big-M)

Decision Variables:

Variable	Type	Meaning
x_ij	Binary {0,1}	1 if job j is assigned to machine i
y_ijj'	Binary {0,1}	1 if job j is processed before job j' on machine i
C_ij	Continuous >= 0	Completion time of job j on machine i
T	Continuous >= 0	Cycle time (makespan of one MPS) — minimized

2.2 MIP Model

Objective:

```
min T
```

C1 — Job assignment (one machine per stage):

```
SUM_{i in M_s} x_ij = 1 for all j, for all s
```

Each job is assigned to exactly one machine per stage. Jobs that bypass a stage have p_ij = 0 for all i in M_s, so the constraint holds with no processing time.

C2 — Disjunctive constraints (no machine overlap):

```
C_ij >= C_ij' + p_ij * x_ij - L*(1 - y_ijj') for all i, j != j' C_ij' >=
C_ij + p_ij' * x_ij' - L * y_ijj' for all i, j != j'
```

No two jobs overlap on the same machine. If y_ijj' = 1 then job j precedes job j' on machine i.

C3 — Stage precedence (flow constraint):

```
C_ij >= C_i'j + p_ij * x_ij for all j, s, i in M_s, i' in M_{s-1}
```

A job cannot start at stage s before completing stage s-1.

C4 — Buffer capacity constraint:

```
#{j : C_i'j <= t < C_ij} <= b_s for all s, all t
```

At no point in time may more jobs reside in the buffer between stages s and s+1 than the capacity b_s. This is linearized using time-indexed binary variables.

C5 — Cycle time definition:

```
T >= C_ij for all i, j where x_ij = 1
```

The cycle time must be at least as large as the latest completion time.

C6 — Bottleneck lower bound:

```
T >= W_i = SUM_j p_ij for all i
```

The cycle time cannot be smaller than the total workload on any single machine.

2.3 Model Size & Complexity

Component	Count
Binary variables x_ij	m x n
Binary sequencing variables y_ijj'	m x n x (n-1)
Continuous variables C_ij	m x n
Disjunctive constraints C2	O(m * n^2)
Buffer constraints C4	O(S * T_horizon)

Even for modest instances — e.g., m = 5 machines and n = 10 jobs — the number of binary sequencing variables reaches 450, and the disjunctive constraints produce a very weak LP relaxation. The buffer constraints are particularly difficult to linearize tightly.

2.4 Critical Assessment of the MIP

Strengths:

- Provides an exact mathematical representation; guarantees optimality given enough time.
- The bottleneck lower bound (C6) tightens the LP relaxation.
- Flexible: alternative objectives (WIP, tardiness) can be substituted directly.
- Useful as a benchmark to evaluate heuristic solution quality on small instances.

Weaknesses:

- Disjunctive constraints (C2) produce a very weak LP relaxation, causing enormous branch-and-bound trees even for small instances.
- Buffer constraints (C4) require time-indexing or non-linear expressions, neither of which is computationally efficient.
- The cyclic nature of the schedule requires additional wrap-around constraints not naturally captured by the static MIP.
- The bypass feature interacts subtly with buffer constraints, complicating the model further.
- The MIP does not scale to industrial instances with dozens of stages and hundreds of job types.

Overall verdict: The MIP is theoretically sound but practically intractable for real-world instances. This directly motivates the heuristic FFLL approach.

3. The FFLL Algorithm

3.1 Algorithm Description

The Flexible Flow Line Loading (FFLL) algorithm solves the problem in three sequential phases, each handling one core decision: who processes what, in what order, and when.

Phase 1: Machine Allocation (LPT Heuristic)

Assign each job to exactly one machine per stage to balance workloads. The Longest Processing Time (LPT) heuristic is applied independently at each stage that has parallel machines:

1. Sort all jobs in decreasing order of their processing time at that stage.
2. Assign each job to the machine at that stage with the currently lowest total workload.
3. Repeat until all jobs are assigned.

Phase 2: Sequencing (Dynamic Balancing Heuristic)

Determine the order in which jobs are released into the system. The key quantities are:

W_i = SUM_j p_ij — Total workload of machine i across the full MPS

W = SUM_i W_i — Total workload of the entire system

p_k = SUM_i p_ik — Total workload imposed on the system by job k

o_ij = p_ij - p_j * W_i / W — Overload contribution of job j to machine i

O_ij = SUM_{k in S_j} o_ik — Cumulative overload on machine i after position j in sequence

The heuristic minimizes:

```
SUM_{i=1}^{m} SUM_{j=1}^{n} max(O_ij, 0)
```

via a greedy procedure: at each position, select the unscheduled job that minimizes the total positive cumulative overload at that point.

Phase 3: Timing of Releases (Bottleneck Anchoring)

1. Identify the bottleneck machine i* (highest workload W_i*). Cycle time T* = W_i*.
2. Let all jobs enter the system as fast as possible (back-to-back releases).
3. Fix the start/completion times of all jobs on the bottleneck machine (no idle time).
4. For upstream machines: delay processing as much as possible — reduces WIP.
5. For downstream machines: process as early as possible — clears jobs quickly.

3.2 Two Application Examples

The following two examples are original — different from the book's Example 16.3.1.

Example A — 2 Stages, 4 Jobs (No Bypass)

Setup: Stage 1 has 2 parallel machines; Stage 2 has 1 machine; 4 jobs in MPS.

	Job 1	Job 2	Job 3	Job 4
Stage 1	5	8	3	6
Stage 2	4	2	7	5

Phase 1 (LPT at Stage 1): Sort descending: Job2(8), Job4(6), Job1(5), Job3(3). Assign: Job2→M1(8), Job4→M2(6), Job1→M2(11), Job3→M1(11). Both machines balanced at load 11. Machine 3 (Stage 2) takes all jobs, W3=18. **Bottleneck: Machine 3, T*=18.**

Phase 2 (Dynamic Balancing): W=(11,11,18), W_total=40, p_k=(9,10,10,11). After computing o_ij values and greedy selection, the final sequence is: **1** → **3** → **2** → **4**.

Phase 3 (Timing): Bottleneck Machine 3 processes jobs with no idle time. Cycle time achieved: **T = 18** (equals lower bound — optimal).

Example B — 3 Stages, 5 Jobs, with Bypass

Setup: Stage 1 has 1 machine; Stage 2 has 2 parallel machines; Stage 3 has 1 machine; 5 jobs. Jobs 2 and 5 bypass Stage 2 (processing time = 0).

	Job 1	Job 2	Job 3	Job 4	Job 5
Stage 1	3	5	2	4	6
Stage 2	4	0	6	3	0
Stage 3	2	3	1	4	2

Phase 1 (LPT at Stage 2): Only Jobs 1, 3, 4 need Stage 2. Sort: Job3(6), Job1(4), Job4(3). Assign: Job3→M2(6), Job1→M3(4), Job4→M3(7). W=(20,6,7,12), W_total=45. **Bottleneck: Machine 1 (Stage 1), T*=20.**

Phase 2 (Dynamic Balancing): After computing all o_ij values and greedy position-by-position selection, the final sequence is: **2** → **1** → **3** → **5** → **4**.

Phase 3 (Timing): Bottleneck Machine 1 (Stage 1) processes jobs with no idle time. Jobs 2 and 5 bypass Stage 2 directly. Cycle time achieved: **T = 20** (equals lower bound — optimal).

3.3 Critical Assessment of the FFLL Heuristic

Strengths:

- Computationally efficient: O(n log n) for Phase 1, O(n^2 * m) for Phase 2 — polynomial overall.
- Three-phase structure mirrors the natural hierarchy of decisions in the problem.
- Dynamic Balancing has a strong theoretical basis: minimizing burst loading reduces buffer occupancy.
- Bypass is handled naturally — bypassed stages contribute zero to overload values.
- Extensive experiments confirm its value as a practical scheduling tool (per the textbook).

Limitations:

- Phases are solved independently and sequentially — no feedback loop between allocation, sequencing, and timing.
- LPT does not always achieve optimal load balance (explicitly noted in the book for Example 16.3.1).
- Dynamic Balancing is greedy with no lookahead — early poor choices cannot be corrected.
- Cycle time minimization is the primary goal; WIP minimization is addressed only indirectly via Phase 3.
- Assumes a fixed, known MPS — not suited for dynamic job arrivals or uncertain processing times.

4. Improved Algorithm: Iterated Greedy with Local Search (IG-LS)

4.1 Design Rationale

The core weakness of the FFLL algorithm is its greedy, non-backtracking sequencing phase. Once a job is placed in the sequence, it stays there regardless of how subsequent choices unfold. Our IG-LS algorithm keeps Phases 1 and 3 identical to FFLL and replaces Phase 2 with three layers of improvement:

Layer 1 — Better starting solution: Begin with the FFLL Dynamic Balancing sequence as a warm start.

Layer 2 — Local search (2-opt swaps): Try every possible pairwise job swap. Accept any swap that reduces the actual simulated makespan. Repeat until no improving swap exists.

Layer 3 — Iterated perturbation: To escape local optima, randomly remove a contiguous block of jobs and reinsert them at a new random position (destruction-reconstruction). Re-apply local search. Accept only if the new solution improves makespan. Repeat for 150 iterations.

Critically, IG-LS optimizes **actual simulated makespan** rather than the overload proxy used by Dynamic Balancing. This is a stronger and more direct optimization signal.

4.2 Pseudocode

```
IG-LS(instance, n_iter=150, block_size=2): // Phase 1 — identical to FFLL alloc <- LPT_allocate(instance) // Phase 2 — improved sequencing seq <- DynamicBalancing(alloc) // warm start (seq, best_ms) <- LocalSearch2opt(seq, alloc) for k = 1 to n_iter: seq' <- Perturb(seq, block_size) // remove & reinsert block (seq', ms') <- LocalSearch2opt(seq', alloc) if ms' < best_ms: seq <- seq' best_ms <- ms' // Phase 3 — identical to FFLL cycle_time <- BottleneckTiming(seq, alloc) return seq, cycle_time, best_ms
```

4.3 Experimental Results on 10 Random Instances

Ten instances were generated with deliberately varied characteristics — from small 4-job/2-stage problems to large 12-job/5-stage problems, with varying bypass probabilities and numbers of machines per stage. The lower bound (LB) used is the bottleneck machine workload after LPT allocation.

#	Label	Jobs	Stg	LB	FFLL	FFLL gap	IG-LS	IG-LS gap	Improv.
1	Small, no bypass	4	2	9.0	13.00	44.4%	12.00	33.3%	7.7%
2	Small, light bypass	5	2	23.0	26.00	13.0%	25.00	8.7%	3.8%
3	Medium, 3-stage	6	3	32.0	35.00	9.4%	32.00	**0.0%**	8.6%
4	Medium, higher bypass	7	3	38.0	42.00	10.5%	38.00	**0.0%**	9.5%
5	Medium-large, 4-stage	8	4	21.0	43.00	104.8%	34.00	61.9%	20.9%
6	Medium, 3 mach/stage	8	3	34.0	49.00	44.1%	35.00	2.9%	28.6%
7	Large, 4-stage	10	4	46.0	56.00	21.7%	50.00	8.7%	10.7%
8	Large, 5-stage	10	5	49.0	56.00	14.3%	54.00	10.2%	3.6%
9	XL, 4-stage	12	4	66.0	97.00	47.0%	72.00	9.1%	25.8%
10	XL, 5-stage, high bypass	12	5	63.0	81.00	28.6%	73.00	15.9%	9.9%
	AVERAGE					**33.8%**		**15.1%**	**12.9%**

IG-LS improved every single one of the 10 instances. On instances 3 and 4 it achieved the theoretical lower bound — proving optimality. The average gap to the lower bound was reduced from 33.8% (FFLL) to 15.1% (IG-LS), more than halving it, while all runs completed in under 1 second.

4.4 Critical Assessment of IG-LS

Strengths over FFLL:

- Reduces average gap to lower bound from 33.8% to 15.1% — more than halved.
- Achieves proven optimality on 2 out of 10 instances (gap = 0%).
- Improvement is consistent across all instance types, not limited to specific cases.
- Optimizes actual makespan rather than the overload proxy — a more direct signal.
- Remains extremely fast — all 10 instances solved in under 1 second.
- The perturbation mechanism escapes local optima that trap the pure greedy.

Remaining limitations:

- Gap to lower bound is still significant on some instances (e.g., Instance 5: 61.9%). Note: the lower bound itself may be loose, so the true optimality gap could be smaller.
- Greedy acceptance criterion (accept only improvements) may miss better solutions reachable via uphill moves — simulated annealing could help.
- Fixed block size (2) for perturbation; adaptive block sizing could better balance exploration vs. exploitation.
- Like FFLL, phases are still solved independently — a fully integrated metaheuristic could achieve further gains.
- Performance on very large instances (50+ jobs) is untested; local search cost is O(n^2) per iteration.

The full Python implementation (FFLL + IG-LS + experiment runner) is provided as a separate package: src/ffll_igls/.